

Tracking Files and Changes

Understanding Git isn't about memorizing commands—it's about recognizing the **model behind it**. Git doesn't just save the latest state, it keeps **snapshots** of your project. For those who understand the structure, almost all commands become logical.

GIT BASICS



The Basic Model: Three Areas

Before you learn commands, you should understand these three areas – they are the heart of Git.

Working Directory

The files you are currently actively working on.

Staging Area (Index)

A preparation area: here you mark what should be included in the next commit.

Repository

The actual Git history with all saved commits.

The Path of a Change



The most important mindset: **Editing** is not yet **staging** – and staging is not yet **permanently saved**. Only `git commit` truly saves the state in the history.

Git File States

When working with Git, think in terms of **states** rather than commands. Always ask yourself: Are the changes in the working directory, the staging area, or already committed?

untracked	File exists but is not yet tracked by Git
modified	File is being tracked, but has been modified
staged	Changes are marked for the next commit
committed	Changes are already saved in a commit

Adding Files to the Repository

New files are initially **untracked** – Git does not yet know about them. The typical workflow for a new file `app.js`:

01

Check status

`git status` – file appears as *untracked*

02

Stage

`git add app.js` – file is added to the Staging Area

03

Commit

`git commit -m "Add app.js"` – state is saved in the repository history

❏ Technically precise: `git add` moves the file into the **Staging Area** – only `git commit` saves it to the **repository history**.

Saving Changes with Commits

What is a commit?

A commit is a captured project state with a message describing what was changed. It contains **only what was previously staged** – not automatically all changed files.

The big advantage

You can consciously decide which changes belong together.

Example: **Commit separately** a typo fix and a new feature, even if they were created in the same work period. This makes the history much more understandable later on.

Writing Good Commit Messages

A commit message should briefly and clearly explain **what** was changed. The more cleanly it is formulated, the easier the history is to follow later on.

✓ Good

Fix bug in password validation

Add form validation

Rename configuration file

✗ Not so good

Update

Changes

fix

Checking Work Status and History

With Git, it is not enough to just make changes – you also have to be able to **read** what is going on. Three commands form a strong combination in everyday use:



git status

Shows the state of the working directory and the staging area.



git diff

Shows the exact content of the changes – which lines have been modified.



git log

Shows the commit history including hash, author, date, and message.



Reading git status correctly

A typical `git status` output shows several states at once:

app.js – staged

Under "Changes to be committed":
already marked for the next commit.

style.css – modified, not staged

Under "Changes not staged for commit": modified, but not yet marked.

notes.txt – untracked

Under "Untracked files": new, Git is not tracking them yet.

With `git status -s` you get a more compact short form – often more convenient in daily use.

git diff and git log in detail

Question	Command	What it shows
Which files are affected?	<code>git status</code>	Status of all files
Which lines have changed?	<code>git diff</code>	Unstaged changes
What is already staged?	<code>git diff --staged</code>	Staged changes
What commits exist?	<code>git log</code>	Full history
Compact history view	<code>git log --oneline</code>	Hash + message per line

📌 **Tip:** `git log --oneline --graph --decorate --all` provides an especially clear view of the history with branch references.

CHAPTER 3

Undoing changes

Undoing is a topic of its own in Git. The most important basic rule is: **Before you undo something, you must know where the change currently resides.**

Depending on the state, a different tool is required.



git restore: Discarding changes

`git restore` restores content in the working directory and, with options, can also affect the staging area.

Reset working directory

```
git restore app.js
```

Resets the file to the last known state. Unsaved changes will be **lost**.

```
git restore .
```

 resets all files – caution!

Reset staging area

```
git restore --staged app.js
```

Removes the file from the staging area without changing its content. Extremely useful if you accidentally added something with `git add` that does not belong in the next commit.

CHAPTER 4

Branch Management

Branches are among the most important Git concepts. At its core, a branch is a **movable pointer to a commit**. This makes branches lightweight, fast – and indispensable for professional development.

Listing and Creating Branches

Listing Branches

`git branch` lists all existing branches. The `*` indicates the current branch:

```
* main
```

```
feature-login
```

```
fix-header
```

Creating a Branch

```
git branch feature-profilseite
```

Creates a new branch—but you remain **on the current branch**. This is a common pitfall for beginners.

Technically, no project folder is copied; only a new name is created that points to the same commit.

Switching between branches

For switching, `git switch` is the clearer option today (older guides still use `git checkout`).

01

Switch branch

`git switch feature-profilseite` – switches to an existing branch

02

Create and switch

`git switch -c feature-profilseite` – creates the branch and switches to it immediately

When switching, Git changes your working context: the branch pointer updates, and files in the working directory are adjusted if necessary.

Why Branches Are Essential



Develop features in isolation

New features are built on their own branch – `main` remains clean and stable.



Isolate bug fixes

Bug fixes run on separate branches without disrupting ongoing feature development.



Work in parallel

Work on multiple topics simultaneously without mixing everything into a chaotic history.

- ❑ A branch is **not a separate project folder**, but an independent development line. Git switches easily between these lines because branches are internally just pointers to commits.



A Realistic Daily Example

Typical setup with three parallel development lines:

main

Currently stable state – is not edited directly

feature-login

New login feature – own commits, independent of main

fix-navbar

Fixes a display bug in the navigation

Essential Commands Overview

Task	Command
Check status	<code>git status</code>
Stage file	<code>git add datei</code>
Create commit	<code>git commit -m "Nachricht"</code>
Show unstaged changes	<code>git diff</code>
Show staged changes	<code>git diff --staged</code>
Show history	<code>git log --oneline</code>
Discard local changes	<code>git restore datei</code>
Unstage file	<code>git restore --staged datei</code>
List branches	<code>git branch</code>
Create and switch branch	<code>git switch -c name</code>

The Essentials: Structure Before Commands

It's not about knowing as many commands as possible, but understanding the **structure of the system** – that is what's decisive with Git.

Know the three areas

Working directory → Staging Area → Repository

Think in states

Always ask yourself: Where are the changes right now?

Use branches

They are not a luxury, but a core tool of professional development